Lab-15 : Advanced MongoDB Concepts

Index | Cursor | Schema Validation | Backup & Restore | Create User

1. Index

An index is a special data structure that improves the speed of searching documents in a collection.

Without index → MongoDB scans all documents (performs a COLLECTION SCAN)
With index → MongoDB directly finds matching documents.

Feature	RDBMS (SQL)	MongoDB
Index concept	CREATE INDEX on a table	db.collection.createIndex()
Default index	Primary Key (auto)	_id field (auto)
Check indexes	SHOW INDEX FROM table	db.collection.getIndexes()
Remove index	DROP INDEX	db.collection.dropIndex()

Remember: The _id field is always indexed automatically. You do NOT need to create an index on _id manually.

Types of Indexes

- Single Field Index — created on one field
- Compound Index — created on two or more fields
- Default _id Index — automatically created on _id for every collection

Ex. Create a single-field index on the company field.

Ans. db.Stock.createIndex({company : 1})

Ex. Create a compound index on sector and sales.

Ans. db.Stock.createIndex({sector:1 , sales:1})

Ex. List all indexes on the Stock collection.

Ans. db.Stock.getIndexes()

Ex. Drop the index on the company field.

Ans. db.Stock.dropIndex({ company : 1 })

2. Cursor

A cursor is a pointer to the result set of a query.

When find() is executed, MongoDB does not return all documents immediately. Instead it returns a **cursor object** which retrieves documents in batches.

Key Concept: db.students.find() does NOT immediately return all documents. It returns a cursor object. The cursor fetches documents in batches (default 101 documents per batch).

Imagine your collection has **10 lakh (1,000,000) documents**. If MongoDB returned everything at once:

- Very high memory usage
- Slow performance — waiting for millions of documents
- Possible system crash
- Network overload

Real Example:
When we open **Instagram Reels**, it does not load all reels at once.
Instead, reels are loaded in batches. As we scroll down, new reels are fetched from the next batch.

Similarly, in MongoDB, a cursor retrieves query results in batches instead of loading all documents at once.

Cursor Methods:

Method	Description	Example
cursor.next()	Returns the next document from the result	myCursor.next()
cursor.hasNext()	Returns true if more documents exist	myCursor.hasNext()
cursor.forEach()	Loops through all documents	myCursor.forEach(printjson)
cursor.toArray()	Converts all cursor results into an array	myCursor.toArray()

cursor.count()	Returns the total number of matching docs	myCursor.count()
cursor.limit(n)	Returns only first n documents	myCursor.limit(5)
cursor.skip(n)	Skips first n documents	myCursor.skip(2)
cursor.sort()	Sorts the result	myCursor.sort({ field : -1 })

Ex. Create a cursor and iterate over all documents.

Ans. var s = db.Stock.find()

s.forEach(printjson) OR **s** (cursor variable)

Here, **var** is used to declare a variable.

var s = db.Stock.find(), s stores the cursor object returned by find().

s.forEach(printjson) - Explicitly loops through cursor.

OR

s (cursor varibale) - Internally it behaves like iterating the cursor.

Ex. Limit cursor to first 3 documents.

Ans. var s = db.Stock.find().limit(3)

s.forEach(printjson) **OR** s

Ex. Sort cursor by sales in descending order.

Ans. var s = db.Stock.find().sort({ sales : -1 })

s

OR

var s = db.Stock.find()

s.sort({ sales : -1 })

Ex. Convert the cursor to an array and return all documents.

Ans. var s = db.Stock.find()

var arr = s.toArray()

printjson(arr) **OR** arr

3. Schema Validation

Schema Validation allows you to enforce rules on documents — ensuring that every document inserted or updated follows a specific structure.

RDBMS Analogy: In SQL, a table's structure (column names, data types, NOT NULL constraints) enforces data integrity automatically. Schema Validation gives MongoDB similar control.

Key Components of $jsonSchema

Component	Meaning	Example Value
bsonType	Data type of the field	"string", "int", "double", "bool", "date", "array", "object"
required	List of mandatory field names	["name", "age"]
properties	Validation rules per field	{ name: { bsonType: "string" } }
minimum	Minimum numeric value	1
maximum	Maximum numeric value	100
minLength	Minimum string length	3
maxLength	Maximum string length	30
enum	Allowed set of values	["active", "inactive"]
additionalProperties	Block fields not in properties	false

Ex. Create a collection named "Companies" with schema validation to ensure that each document must contains a company field (string) and a sector field (string).

```
db.createCollection("Companies", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["company", "sector"],
   properties: {
    company: {
     bsonType: "string",
     description: "must be a string and is required"
    },
    sector: {
     bsonType: "string",
     description: "must be a string and is required"
    }
   },
   additonalProperties: false
  }
 }
})
```

means the document must be stored as an object (MongoDB document)

No extra field allowed

Ex. Create a collection named "Student" where each student has name and address are embedded document and mobilenumber and emailaddress are multivalued field that stores an array of values.
(example of embedded document and multivalued field)

```
db.createCollection("Student", {
 validator: {
  $jsonSchema: {
   bsonType: "object",
   required: ["name", "address", "mobilenumber", "emailaddress"],
   properties: {
    name: {
     bsonType: "object",
     required: ["firstname", "lastname"],
     properties: {
      firstname: { bsonType: "string" },
      lastname: { bsonType: "string" }
     }
    },
    address: {
     bsonType: "object",
     required: ["city", "state"],
     properties: {
      city: { bsonType: "string" },
      state: { bsonType: "string" }
     }
    },
    mobilenumber: {
     bsonType: "array",
     items: { bsonType: "string" }
    },
    emailaddress: {
     bsonType: "array",
     items: { bsonType: "string" }
    }
   }
  }
 }
})
```

4. Backup & Restore

MongoDB backup and restore operations are performed using two command-line tools that are run in the Windows/Linux terminal (NOT inside the MongoDB shell):

Tool	Purpose	Direction
mongodump	Takes a backup of database/collection	MongoDB → Backup Files
mongorestore	Restores data from a backup	Backup Files → MongoDB

Important Flags / Parameters:

Flag (Short)	Flag (Full)	Meaning
-d	--db	Specify the database name
-c	--collection	Specify the collection name
--out	--out	Location where backup will be stored (mongodump only)
path	path	Source path to restore from (mongorestore only)

<u>BACKUP using mongodump:</u>

<u>Syntax:</u>

```
mongodump --db <database_name> --collection <collection_name> --out D:\backup

                              OR

mongodump -d <database_name> -c <collection_name> --out D:\backup
```

Backup entire database (all collections)

```
mongodump  --db <database_name> --out D:\backup
```

Ex. Backup Stock collection from Darshan database.

mongodump -d Darshan -c Stock --out D:\backup

After running the command, MongoDB creates this folder structure automatically.

D:\backup

→ Darshan

→ Stock.bson (**Autual data – binary format**)

Stock.metadata.json (**collection metadata – indexes,etc..)**

Restore using mongorestore:

Syntax

mongorestore --db <database_name> --collection <collection_name> **path**

Ex. Restore Stock collection from backup:

mongorestore --db Darshan --collection Stock D:\backup\Darshan\Stock.bson

Ex. Restore entire Darshan database.

mongorestore --db Darshan D:\backup\Darshan

5. Create User & Add Role

In a production MongoDB system, you don't want everyone to have full access to the database. MongoDB allows you to create users with specific roles — each user can only perform operations their role allows.

- A role in MongoDB grants privileges to perform specific operations on given resources.
- Users are created using the createUser() method.
- If the user already exists, createUser() returns an error.

Built-in Roles:

Role	Permissions	Use Case
read	Read-only access to a database	Reporting, analytics
readWrite	Read and write access to a database	Application users
dbAdmin	Admin tasks (indexes, stats) — no read/write	Database administrators
userAdmin	Create/manage users in a database	User management
dbOwner	All operations on the database	Full database owner
root	Superuser — access to everything	System administrator

Syntax – createUser():

use <database_name> - First switch to the target database

```
db.createUser( {
    user:  "username",
    pwd: "password",
    roles: [
            {role : "roleName" , db: "databaseName"}
          ]
    } )
```

Ex. Create a read-only user 'abc' on the Darshan database:

Ans.

```
use Darshan

db.createUser ({
        user:"abc",
        pwd:"123",
        roles:
                [
                {role:"read", db:"Darshan"}
                ]
})
```

List All User:

use admin
db.system.users.find() OR **show users**

Delete a User – dropUser():

db.dropUser("username")